Clase 3

@September 7, 2026

Introducción

Hoy avanzamos en el núcleo del motor gráfico con los **Shaders** y la arquitectura de renderizado con DirectX 11. El objetivo del primer parcial es renderizar un triángulo en pantalla, para posteriormente escalar a objetos como cubos y abstraer la lógica en clases reutilizables.

Desarrollo

Shader Triangle.hlsl

Un **shader** es un programa especializado que se ejecuta en la GPU y define cómo se procesan y muestran los gráficos en pantalla. Creamos el archivo Triangle.hlsl en la carpeta de recursos.

Este archivo define la estructura de entrada de los vértices (**VSInput**), la estructura intermedia que pasa del Vertex Shader al Pixel Shader (**PSInput**), y las dos funciones principales (`VSMain` para vértices y `PSMain` para píxeles).

El shader tiene dos partes principales:

Vertex Shader (`VSMain`)

Procesa la geometría y los vértices. Recibe la posición en 3D y el color de cada vértice, convierte la posición a coordenadas homogéneas (float4) y la pasa a la siguiente etapa.

Pixel Shader (`PSMain`)

Se encarga de la rasterización y el dibujado de píxeles. Rellena los espacios entre vértices interpolando el color y devuelve el color final hacia la pantalla.

Header Engine.h

En Engine.h definimos la interfaz pública del motor utilizando buenas prácticas de C++ moderno:

Las 4 Funciones Fundamentales

Aquí declaramos las funciones con un noexcept para que sea obligatorio rellenarlas.

```
extern "C" {
    ENGINE_API bool
    Engine_Initialize(HWND hand, int width, int height) noe
xcept;

    ENGINE_API void
    Engine_Update() noexcept;

    ENGINE_API void
    Engine_Render() noexcept;

    ENGINE_API void
    Engine_Shutdown() noexcept;
}
```

> ! `Initialize` regresa un `bool` para notificar al ejecutable si la tarjeta gráfica o la ventana fallaron al crearse en el arranque. Las funciones `Update`, `Render` y `Shutdown` son `void` porque se ejecutan continuamente o limpian recursos sin necesidad de reportar un estado de éxito inicial.

Deshabilitar copia y movimiento

Se eliminan explícitamente los constructores de copia para evitar que se creen duplicados accidentales del motor.

```
Engine(const Engine&) = delete;
Engine& operator=(Engine&&) = delete;
```

Patrón PImpl (Pointer to Implementation)

Se declara una estructura interna y un puntero privado. Esto **oculta** todas las dependencias y librerías de DirectX 11 del archivo de cabecera público, acelerando los tiempos de compilación del Sandbox.

```
private:
    struct Implementation;
    Implementation* m_implementation = nullptr;
```

Implementación Interna y Recursos de DirectX (Engine.cpp)

Dentro de `Engine.cpp` se implementa la estructura `Implementation`, la cual almacena los recursos nativos de DirectX 11:

`ID3D11Device* device`

Crea y gestiona los recursos de la GPU en memoria (buffers, shaders, texturas).

`ID3D11DeviceContext* context`

Ejecuta y envía las órdenes de renderizado a la GPU.

`IDXGISwapChain* swapChain`

Gestiona el intercambio de buffers entre la memoria de video y la pantalla (evita el parpadeo de imagen usando un *backbuffer*).

`ID3D11RenderTargetView* renderTarget`

Representa el destino de renderizado en pantalla.

Compilación Dinámica de Shaders

Creamos la función `CompileShader` para cargar y compilar archivos HLSL en tiempo de ejecución utilizando `D3DCompileFromFile`. Incluye un bloque de preprocesador (`#ifdef _DEBUG`) para aplicar banderas de depuración en modo Debug y optimizaciones máximas (`D3DCOMPILE_OPTIMIZATION_LEVEL3`) en publicaciones finales.

Conclusión

No manches, hubieron demasiadas partes en las que genuinamente no comprendí que dijo el profe y tuve que repetir el video varias veces. Aun así, cuando ya entré en la vibra como que ya comprendí su vocabulario y forma de

referirse a las cosas. Aunque se que en unos meses cuando vuelva a leer estas notas, esas palabras no van a tener significado para mi.

Glosario

HLSL y Shaders

Uso de los dos puntos `:` en HLSL

Indican una **semántica**. Es una etiqueta requerida por DirectX para saber qué significado físico o de pipeline tiene una variable (por ejemplo, saber si un dato representa una posición en el espacio o un color).

```
struct VSInput
{
    float3 position : POSITION;
    float4 color : COLOR;
};
```

Diferencia de colores en HLSL

- *Morado en minúsculas (position, color):* Son los **nombres de las variables** definidos por el programador.
- *Verde en mayúsculas (**POSITION**, **COLOR**, **SV_POSITION**, **SV_Target**):* Son las **semánticas de HLSL** (palabras clave del sistema que le dicen a la GPU para qué sirve esa variable dentro del pipeline gráfico).

Pixel Shader

Etapa del pipeline gráfico que calcula el color final de cada píxel individual dentro de los polígonos dibujados en pantalla.

Vertex Shader

Etapa inicial que procesa la posición, orientación y atributos de cada vértice.

Conceptos de C++ y Arquitectura de Software

`extern "C"`

Le indica al compilador de C++ que no aplique el *name mangling* (alteración de nombres de funciones). Permite que las funciones exportadas por la DLL mantengan su nombre original en C, facilitando su llamado desde otros proyectos o lenguajes.

ENGINE_API

Es una macro definida mediante preprocesador (`#define`). Cuando se compila el motor gráfico, se expande a `__declspec(dllexport)` (para exportar funciones de la DLL). Cuando la lee el Sandbox, se expande a `__declspec(dllimport)` (para importar las funciones de la DLL).

Handle (y tipos como `HWND`)

Un *Handle* (identificador) es un puntero opaco o entero que el sistema operativo asigna para hacer referencia a un recurso del sistema. La `H` inicial al principio de tipos de Windows API significa "Handle"; por ejemplo, `HWND` significa *Handle to a Window* (identificador de ventana).

`void` y `void*`

- `void`: Indica que una función no regresa ningún valor (por ejemplo, `void Render()`).
- `void*`: Es un **puntero genérico** a memoria sin tipo definido. Se usó en `Initialize(void* nativeWindow)` para pasar el identificador de ventana de Windows de forma neutra sin incluir librerías del sistema en la cabecera.

`noexcept`

Garantiza al compilador que una función **no lanzará excepciones** de C++. Permite que el compilador genere código ejecutable más rápido y optimizado.

Punteros ()

Variables que guardan la dirección de memoria donde reside otro objeto o variable. Se declaran con un asterisco.

```
ID3D11Device* device
```

Operador de Referencia y Dirección (Ampersand `&`)

- *En tipos de parámetros (`const Engine&`):* Significa **paso por referencia**; pasa el objeto original sin hacer copias pesadas en memoria.
- *Frente a una variable (`&errors`):* Es el operador **"dirección de". Extrae la posición de memoria de esa variable.**

Macro para liberar puntero (`SAFE_RELEASE` / `SafeRelease`)

En DirectX, los recursos COM deben liberarse manualmente. La función/macro verifica `if (object != nullptr)`, llama a `object->Release()` y luego asigna `object = nullptr;` para evitar liberar memoria que ya fue destruida (*dangling pointers*).

Diagonal invertida (`\`) en macros

Es el caracter de **continuación de línea**. Indica al precompilador que la definición de la macro continúa en el renglón de abajo.

Directiva `#ifdef`

Instrucción de compilación condicional. `#ifdef _DEBUG` revisa si estamos en modo Depuración en Visual Studio; de ser así, compila bloques de código específicos para detección de errores que no estarán presentes en la versión final del juego.